KuCoin candle data (Spot): exact “how-to”

You can pull historical candles from KuCoin **for free** using the **public REST** endpoint:

- **GET** `https://api.kucoin.com/api/v1/market/candles`

This is the simplest reliable way to build a training dataset.

---

1) Candle endpoint basics

Required query params

- `symbol` — e.g. `BTC-USDT`
- `type` — timeframe string (examples below)

Optional query params (for history/backfill)

- `startAt` — Unix timestamp in **seconds** (inclusive)
- `endAt` — Unix timestamp in **seconds** (inclusive)

Response format

The `data` field is an array of arrays, each candle shaped like:

```
[
  startTime,   // candle start time (unix seconds, as a string)
  open,
  close,
  high,
  low,
  volume,
  turnover     // quote volume / transaction amount
]
```

Ordering

KuCoin returns candles **newest → oldest** (reverse chronological). You almost always want to reverse them before saving.

Per-request limit

Each request returns **up to 1500 candles**.

---

2) Supported timeframe strings

Common ones you'll use:

- `1min`
- `3min`
- `5min`
- `15min`
- `30min`
- `1hour`
- `2hour`
- `4hour`
- `6hour`
- `8hour`
- `12hour`
- `1day`
- `1week`

For your project: - **Daily**: `type=1day` - **Hourly**: `type=1hour`

---

3) Backfill strategy (paging by time)

Because you only get 1500 candles per call, you page through time.

Idea

1) Pick a **target range** `[startAt, endAt]`. 2) Request candles for that range. 3) Read the **oldest candle** returned. 4) Set the next request's `endAt` to **(oldest_start_time - 1)**. 5) Repeat until you've reached `startAt`.

Window size cheat

If you want maximum efficiency, choose a window that covers ~1500 candles:

- `1day` window ≈ `1500 * 86400` seconds ≈ **1500 days**
- `1hour` window ≈ `1500 * 3600` seconds ≈ **62.5 days**

But you don't have to be exact—paging by `endAt` works regardless.

---

4) Minimal Python (no SDK) — backfill daily BTC-USDT

```
import time
import requests
```

```
import csv

BASE = "https://api.kucoin.com/api/v1/market/candles"

TF_TO_SECONDS = {
    "1min": 60,
    "3min": 3*60,
    "5min": 5*60,
    "15min": 15*60,
    "30min": 30*60,
    "1hour": 3600,
    "2hour": 2*3600,
    "4hour": 4*3600,
    "6hour": 6*3600,
    "8hour": 8*3600,
    "12hour": 12*3600,
    "1day": 86400,
    "1week": 7*86400,
}

def fetch_chunk(symbol: str, tf: str, start_at: int, end_at: int):
    params = {"symbol": symbol, "type": tf, "startAt": start_at, "endAt":
end_at}
    r = requests.get(BASE, params=params, timeout=30)
    r.raise_for_status()
    j = r.json()
    data = j.get("data", [])
    # data is newest->oldest
    return data

def backfill(symbol="BTC-USDT", tf="1day", start_at=None, end_at=None,
out_csv="btc_usdt_1day.csv"):
    assert tf in TF_TO_SECONDS
    now = int(time.time())

    if end_at is None:
        end_at = now
    if start_at is None:
        # default: ~10 years back
        start_at = now - 10*365*86400

    rows_written = 0
    current_end = end_at

    with open(out_csv, "w", newline="", encoding="utf-8") as f:
        w = csv.writer(f)
        w.writerow(["timestamp", "open", "high", "low", "close", "volume",
"turnover"])
```

```
        while current_end > start_at:
            # request a large window (optional); you can just use
startAt=start_at each time too
            window = 1500 * TF_TO_SECONDS[tf]
            current_start = max(start_at, current_end - window)

            data = fetch_chunk(symbol, tf, current_start, current_end)
            if not data:
                # no candles in this region; step back by one window
                current_end = current_start - 1
                continue

            # reverse to oldest->newest for writing
            data.reverse()

            for c in data:
                ts, op, cl, hi, lo, vol, turnover = c
                w.writerow([int(ts), float(op), float(hi), float(lo), float(cl),
float(vol), float(turnover)])
                rows_written += 1

            # set next page end to just before the oldest candle we got
            oldest_ts = int(data[0][0])
            current_end = oldest_ts - 1

            # be polite
            time.sleep(0.2)

    return rows_written

if __name__ == "__main__":
    rows = backfill(symbol="BTC-USDT", tf="1day",
out_csv="BTCUSDT_1day_kucoin.csv")
    print("rows:", rows)
```

What this produces: - a single CSV where `timestamp` is candle start time (UTC) - one row per day

---

5) Minimal Node.js (no SDK) — fetch one chunk

```
const BASE = "https://api.kucoin.com/api/v1/market/candles";

async function fetchKlines({ symbol = "BTC-USDT", type = "1day", startAt,
endAt }) {
  const url = new URL(BASE);
```

```
  url.searchParams.set("symbol", symbol);
  url.searchParams.set("type", type);
  if (startAt) url.searchParams.set("startAt", String(startAt));
  if (endAt) url.searchParams.set("endAt", String(endAt));

  const res = await fetch(url);
  if (!res.ok) throw new Error(`HTTP ${res.status}: ${await res.text()}`);
  const json = await res.json();
  return json.data || [];
}

(async () => {
  const endAt = Math.floor(Date.now() / 1000);
  const startAt = endAt - 200 * 86400; // last ~200 days
  const candles = await fetchKlines({ symbol: "BTC-USDT", type: "1day",
startAt, endAt });
  console.log("count:", candles.length);
  console.log("first (newest):", candles[0]);
  console.log("last (oldest):", candles[candles.length - 1]);
})();
```

To backfill in Node, use the same paging rule: - read the **oldest** candle's timestamp - set next `endAt = oldest_ts - 1`

---

6) If you want LIVE candles (optional)

KuCoin also supports a WebSocket stream: - topic: `/market/candles:{symbol}_{type}` - example: `/market/candles:BTC-USDT_1hour`

You'd use WS for *real-time updates*, not backfill.

---

7) Common gotchas

1) **Reverse ordering**: REST returns newest→oldest. Reverse before saving. 2) **Missing intervals**: if there were no trades, KuCoin may not publish a candle for that interval. 3) **Timestamps are seconds**: don't pass milliseconds. 4) **Turnover vs volume**: - `volume` is base asset amount - `turnover` is quote amount

---

8) Recommended output format for your training pipeline

Store as: - `timestamp` (int unix seconds, UTC) - `open, high, low, close` (float) - `volume, turnover` (float)

This matches most ML pipelines and makes resampling/features easy.